Experiment No. 01 Date Page No 01

Name of Experiment GIT REPOSITORY

Q. Demonstrate how to create a project in a remote repository & apply fork, merge, diff, merge conflict, branch, & pull request concepts on the repository using GitHub.

=> Step 1 : Create repository in GitHub
Open GitHub
Click new repository
Enter repo name : git_demo

Step 2 : Clone repo to local system
Open terminal and run :
git clone https://github.com/username/gid_demo.git
cd git_demo

Step 3 : Commit & Push
git add file.txt
git commit -m "Initial Commit"
git push origin main

Step 3 : Create a file & write the content "Hello, GitHub"

Step 5 : Create and Switch to branch
git branch feature1
git checkout feature1

Experiment No. 01 Date Page No 02

Name of Experiment GIT REPOSITORY

Step 6 : Modify file in feature branch
Open file1.text and add a line
Now commit changes
git add file1.txt.
git commit -m "Feature branch update"
git push origin feature1

Step 7 : Show diff b/w branches : git diff main feature1

Step 8 : Create merge conflict
git checkout main
>open file1.txt & add another line
git commit -m "Main branch change"
git merge feature1

Step 9 : Resolve conflict :
>open file1.txt & add another line
git add file1.txt
git commit -m "Merge conflict resolved"
git push origin main

Step 10 : Create pull request
Open repository in github
Click pull requests
Click create pull request to merge
into original repository.

Experiment No. 02 Date Page No 03

Name of Experiment DOCKER CONTAINERIZATION

2. Application Containerization using Docker
Create a Dockerfile to package static web assests into an apache tomcat container, configuring the environment server html content & distributing the finalized image to a container registery.

⇒ Installation of Dockers:

1) Download Dockers Desktop from the official website
https://www.dockers.com/products/dockers-desktop

2) Install Dockers Desktop: Run the installer & follow the instructions displayed on the screen to complete the installation

3) Config Dockers: During installation, enable the WSL2 option if prompted & supported by your system.

4) Dockers Desktop: Open Dockers & wait until Dockers has started successfully

5) Check version: Open cmd & check Dockers version → dockers --version.

Experiment No. 02 Date Page No 04

Name of Experiment DOCKER CONTAINERIZATION

Program :

1) Create a project folder
Create a folder named myapp on the desktop

2) Create an html file & define it's contents

3) Create a dockerfile :

```
FROM tomcat : 10.1-jdk17-temurin
COPY index.html /usr/local/tomcat/webapps/
    myapp/index.html
EXPOSE 8080.
```

Save the file without an extension

4) Open the terminal:

```
docker build -t my-tomcat
docker images
```

5) View the output
http://localhost:8080/myapp

6) Stop the container
docker stop my-tomcat-container.

Experiment No. 03 Date Page No 05

Name of Experiment JENKINS PIPELINE

Q. Continuous Integration pipeline automation using Jenkins:
Design & implement a Jenkins pipeline to automate the CI lifecycle, config webhooks to trigger builds from git & executing automated build stages to ensure code stability.

→ Installation of Jenkins:

1) Install Java (JDK) after installing, verify with java --version in terminal

2) Install Jenkins, select the installation location & install

3) Start Jenkins: Enter https://localhost:8080

4) Copy the initialAdminPassword credentials from C:/ProgramData/Jenkins/.jenkins/secrets/initialAdminPassword & paste it to the jenkins login page.

5) Config Jenkins: Click install suggested plugins, click save & finish.

Step 1: Create Github repository.

Step 2: Config the Jenkins pipeline:- Open Jenkins > new item > Jenkins-CI > Select Freestyle Project.

Experiment No. 03 Date Page No 06

Name of Experiment Jenkins Pipeline

Step 3: Config the freestyle project:
Open the Jenkins job > Click config > select Git paste git repo > select github hook trigger for GITSH polling > Click save.

Step 4 : Config the build command.
Add build step > echo build started > Click save

Step 5: Start Cloudflare tunnel:
Open command prompt > Cloudflare tunnel.
--url http://localhost:8080
Copy the HTTPS url displayed in the terminal.

Step 6: Add the github webhook
Open github repository > Click settings > select webhooks > click add webhook
Payload url > https://tunnel-url/github-webhook/
content type - application/json

Step 7: Test the webhook: Open github repo > add new file or modify an existing file.>
Commit & push the changes. Github sends a webhook request to Jenkins. Jenkins automatically triggers the build.

Step 8: Check the output
Open Jenkins > select the Jenkins-u project
Open Latest build > Click console output

Experiment No. 04 Date Page No 07

Name of Experiment: DOCKER IMAGE WITH JINKINS PIPELINE

Q. Automated Docker Image lifecycle with Jenkins pipelines - Implement a continuous integration workflow that automates the building tagging and pushing of a docker image to docker hub upon the successful completion of the application build stage.

⇒ Step 1 : Verify docker installation: docker --version

Step 2: Create a folder named prog10 on the desktop
Create app.py file in it.

Step 3: Create a dockerfile :
FROM python : 3.10
WORKDIR /app
COPY . .
CMD ["python", "app.py"]

Step 4: Open CMD & build the docker image :
- docker build -t prog10
- docker image
- docker run -rm prog10

Step 5: Tag & push the Docker image to docker hub. docker tag prog10 username/prog10: latest. docker push username/prog10: latest

Experiment No. 04 Date Page No 08

Name of Experiment: Docker Image with Jenkins Pipeline

Step 6: Push the project code to github:
- git init
- git add .
- git commit -m "Initial commit"
- git branch -M main
- git remote add origin https://github.com/
- git push -u origin main

Step 7: Create a Dockerhub access token:
Open Dockerhub → account settings → security → create access token.

Step 8: Add docker hub credentials in Jenkins
Open Jenkins → Manage Jenkins → Credentials → Global → Add credentials
- kind : Secret text
- Secret : Dockerhub access token
- ID : Dockerhub -path

Step 9: Create a Jenkins File

```
pipeline {
    agent any
        environment {
            DOCKER_IMAGE = "myapp/prog10"
    }
    stages {
        stage ('Clone Repository') {
            steps {
                git branch : 'main',
                url : 'Github_Repository-URL'
            }
        }
}
```

Experiment No. 04 Date Page No 09

Name of Experiment: Docker Image with Jenkins Pipeline

```
stage ('Login to Docker Hub') {
    steps {
        withCredentials ([string (
            credentialsId: 'docker-hub-PAT',
            variable: 'Docker-PAT'
        ])}
        bat """
        @ echo off
        echo %DOCKER_PAT% | docker login -u
        %your_dockerhub_username% --password-stdin
        if errorlevel !=0 exit /b 1
        """
    }
}

stage ('Push Docker Image') {
    steps {
        bat 'docker push %DOCKER_IMAGE%:latest'
    }
}
post {
    success {
        echo 'Docker image build & pushed successfully';
    }
    failure {
        echo 'Pipeline failed. Check console output"
    }
}
}
```

Experiment No.....04.....Date.....Page No 10

Name of Experiment...Docker Image with Jenkins Pipeline....

→ Push the Jenkins to github
git add .
git commit -m "Add Jenkins Pipeline"
git push origin main

Step 10: Build the Pipeline
- Open Docker pipeline job
- Click build now
- Wait for the build to complete
- Click the build no
- Open console output.

Experiment No. 05 Date Page No 11

Name of Experiment MAVEN

Q. Automated Build & Dependency Management using Apache Maven.
- Construct a standard Java project structure to manage dependencies via pom.xml & automate the build lifecycle - specifically compilation, unit testing & packaging to generate deployable artifacts

=> 1) Open Jenkins & go to manage Jenkins
→ plugins install the maven integration plugin

2) Create a new maven project > select maven archetype - quick start > com.example > ~~select~~ done

3) Open the created demo folder & check the pom.xml file

4) Create a github actions workflow file inside the project. .github/workflow/maven.yml
- Add one more file inside maven > New. java

5) Add the code to maven.yml :

```
name : Java CI/CD pipeline
on :
  push :
    branches : ["main"]
  pull request :
    branches : ["main"]
  workflow - dispatch :
```

Experiment No. 05 Date Page No 12

Name of Experiment MAVEN

```
jobs:
  build:
    runs-on: windows-latest
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: set up JDK 21
      uses: actions/setup-java@v4
      with:
        java-version: '21'
        distribution: 'temurin'
    - name: Compile Java Program
      run: javac neo.java
```

6) Save the file & push the maven project to a new github repo

7) Open the github repository & click actions. Verify the build is completed successfully.